Lección 5 1: Bucles for, Estructuras de Control y Manejo de Errores

Domina el ciclo `for` para iteración masiva y aprende a controlar el flujo de ejecución mediante gestión de errores y saltos condicionales.

El Bucle for: Iterando Colecciones

Aplicando Operaciones a Miles de Datos Simultáneamente

El ciclo `for` es la herramienta más común para **iterar** (recorrer) elementos de colecciones como listas, tuplas, diccionarios y conjuntos. Permite aplicar la misma operación a cientos o miles de registros de forma automática.

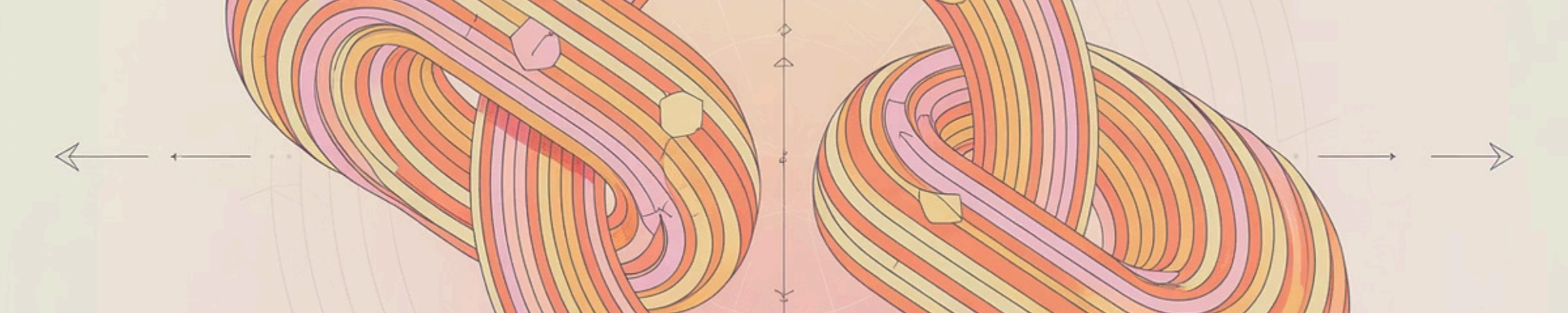
Ejemplo básico: Aplicar IVA

```
lista_precios = [10, 20, 30]
for precio in lista_precios:
    print(precio * 1.21)
```

Iteración en Diccionarios

Por defecto, iterar sobre un diccionario recorre las **claves**. Para acceder a valores o pares clave-valor, utiliza los métodos `.values()` o `.items()`:

```
for clave, valor in mi_dict.items():
 print(f"La {clave} es {valor}")
```



Herramientas de Iteración

Controlando el Alcance y el Rango

Función range()

Genera secuencias numéricas para iterar un número específico de veces:

- `range(fin)`: desde 0 hasta fin-1
- `range(inicio, fin)`: desde inicio hasta fin-1
- `range(inicio, fin, salto)`: con incrementos personalizados

Bucles Anidados

Un bucle `for` dentro de otro, ideal para estructuras de datos complejas.

Uso en Análisis de Datos: Procesar listas de listas, recorrer diccionarios dentro de listas, o analizar datos multidimensionales.

List Comprehensions

Creación de Listas de Forma Rápida y Elegante

Las **comprensiones de listas** ofrecen una sintaxis concisa para crear listas nuevas aplicando expresiones a cada elemento de un iterable, incluyendo filtros condicionales en una sola línea.

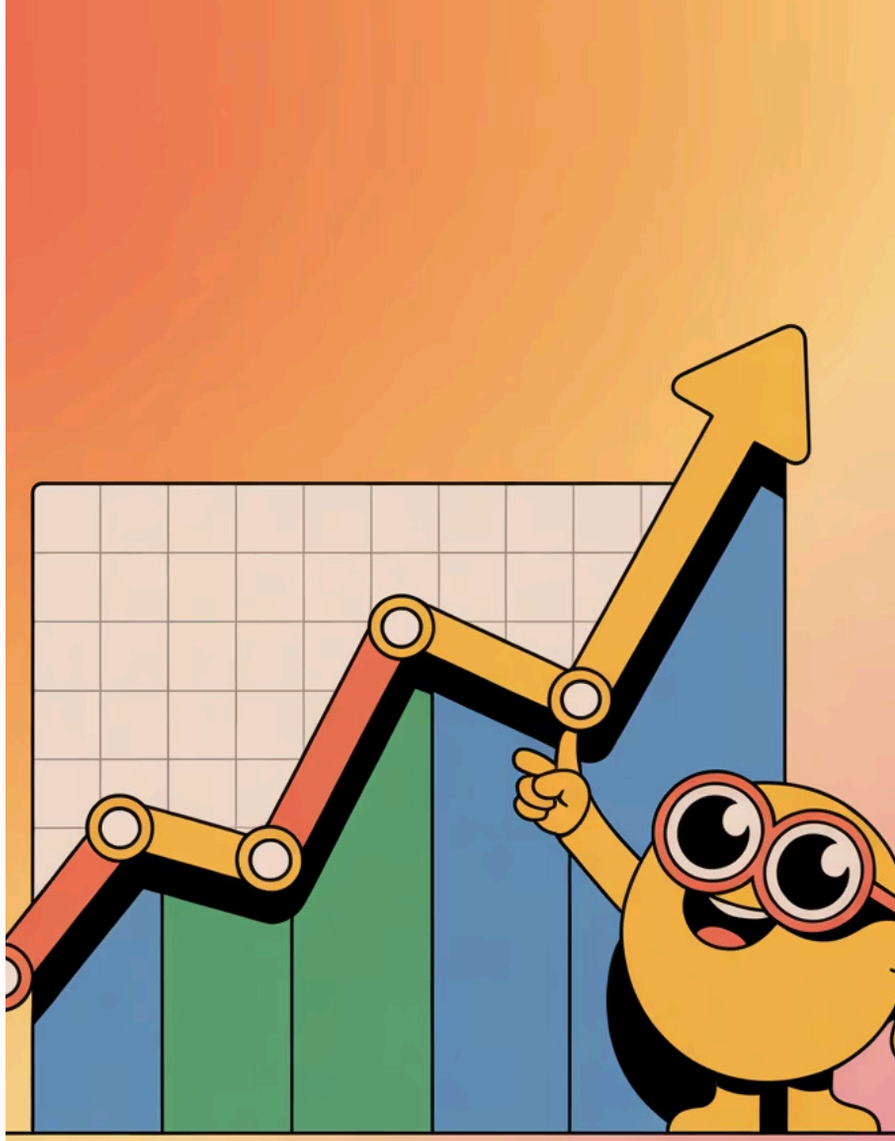
Sintaxis Clásica (Bucle for)

```
nueva = []
for x in lista:
    if x > 5:
        nueva.append(x * 2)
```

Sintaxis List Comprehension

```
nueva = [x * 2 for x in lista
        if x > 5]
```

Enfoque en Análisis de Datos: Las list comprehensions son cruciales para la **eficiencia** y legibilidad del código. Permiten transformar datos y crear nuevas columnas en DataFrames con una sintaxis clara y compacta.



Python Code Optimization

Controlando el Flujo del Bucle

Ajustando la Ejecución con break, continue y pass

break

Detiene el ciclo inmediatamente y salta a la primera línea después del bucle.

```
if elemento == "Dato Buscado":
    break
```

Uso DA: Buscar un registro específico. Una vez encontrado, el bucle se detiene para ahorrar procesamiento.

continue

Detiene la iteración actual y salta a la siguiente inmediatamente.

```
if dato is None:
    continue
```

Uso DA: Limpieza de datos. Ignora valores nulos o defectuosos sin detener el proceso completo.

pass

No ejecuta ninguna acción. Funciona como marcador de posición donde se requiere código sintácticamente.

Útil durante el desarrollo para definir estructuras que completarás más tarde.

Control de Errores: try-except

Blindando Nuestro Código contra Fallos Inesperados

El manejo de excepciones es fundamental para evitar que programas se detengan abruptamente ante errores inesperados como divisiones por cero o conversiones de tipo incorrectas.

01

Bloque try

Contiene el código que podría generar un error

02

Bloque except

Se ejecuta SÓLO si ocurre el error especificado

03

Bloque else

Se ejecuta SÓLO si el try fue exitoso

04

Bloque finally

Se ejecuta SIEMPRE, haya error o no

```
try:
    resultado = num1 / num2
except ZeroDivisionError:
    resultado = 0
else:
    print("Operación exitosa")
finally:
    print("Proceso terminado")
```

Enfoque en Análisis de Datos: Esencial para procesar grandes conjuntos de datos con "datos sucios". Permite registrar errores o asignar valores predeterminados sin detener el análisis completo.